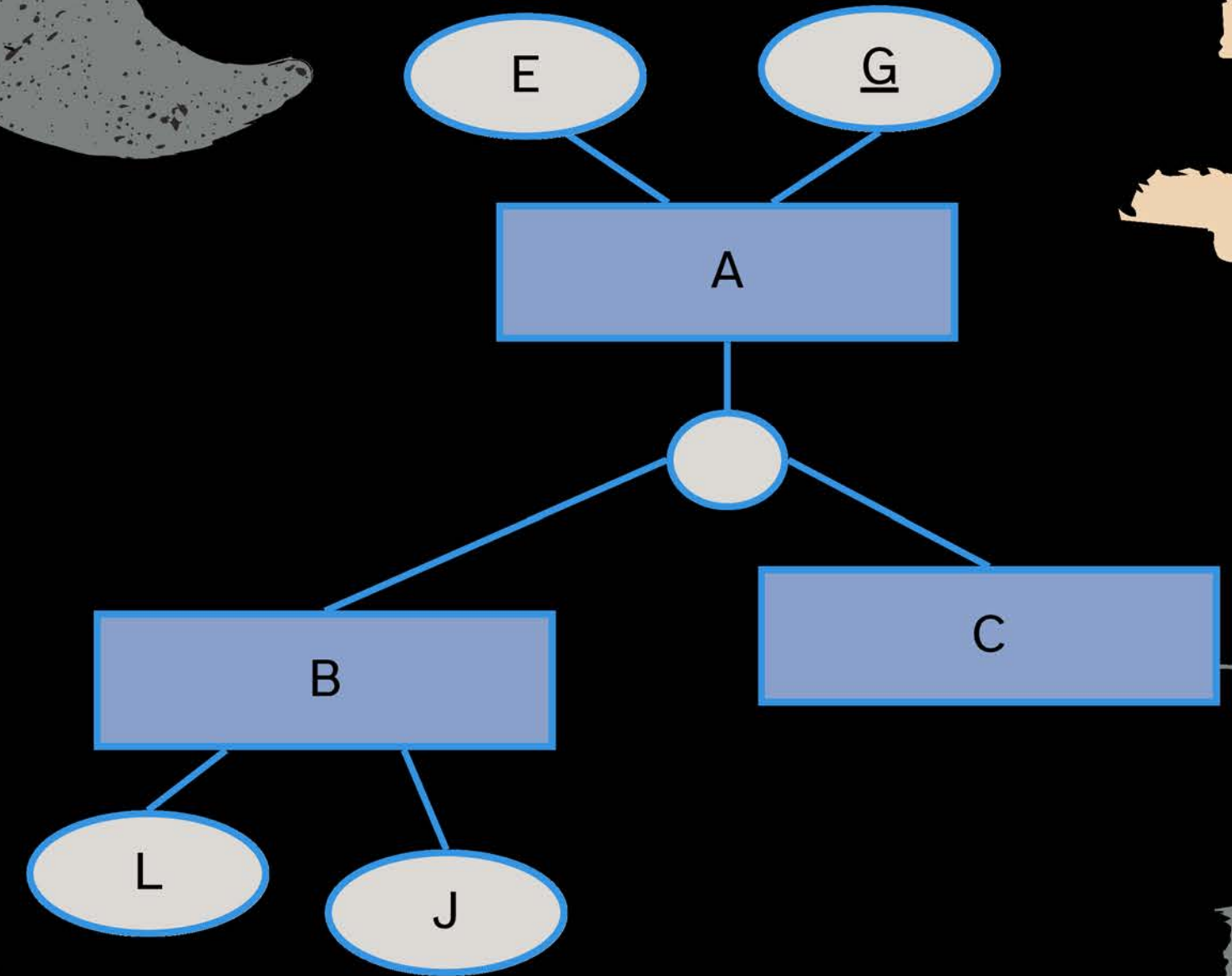
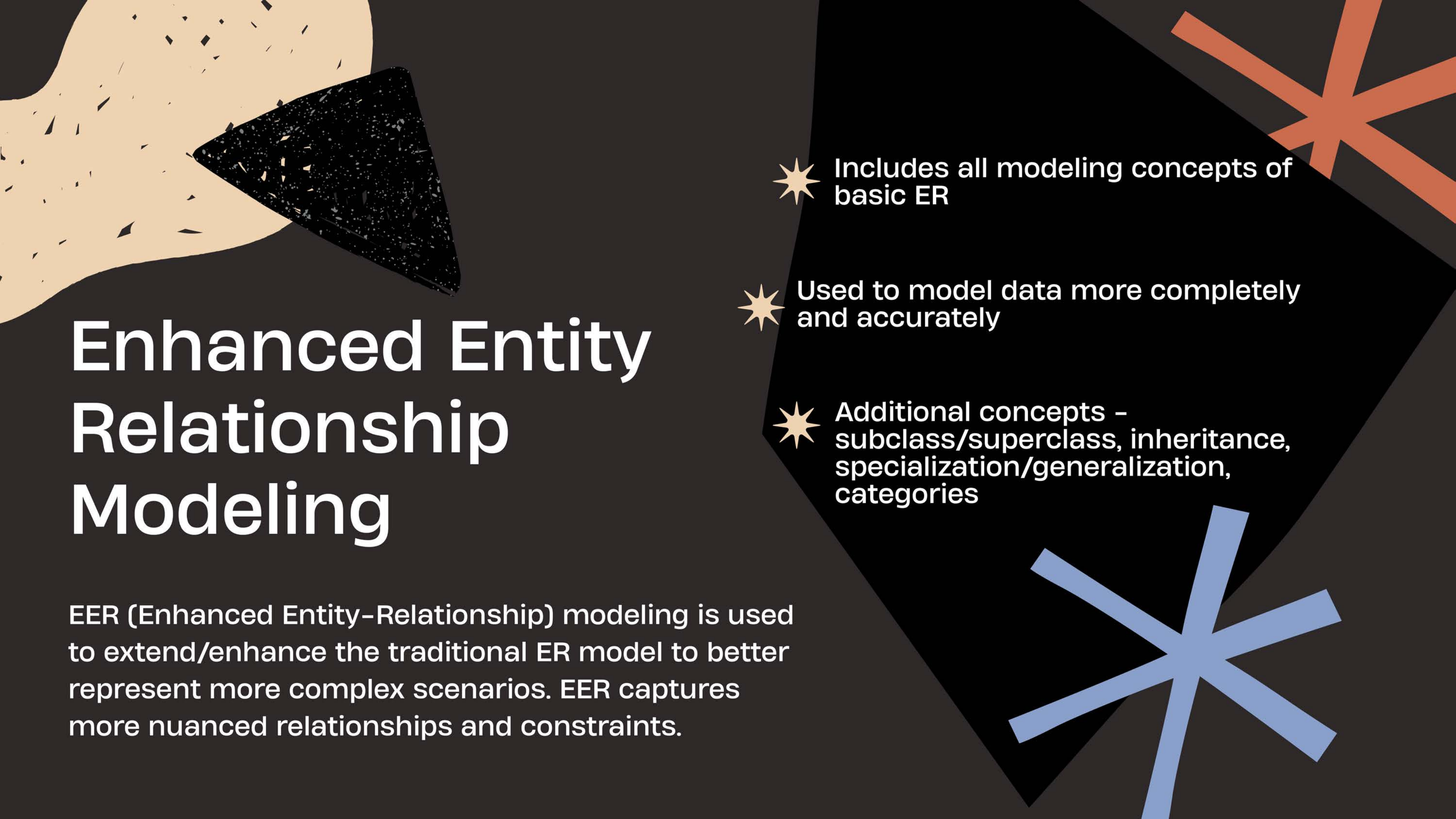


Lecture 3: Enhanced Entity Relationships





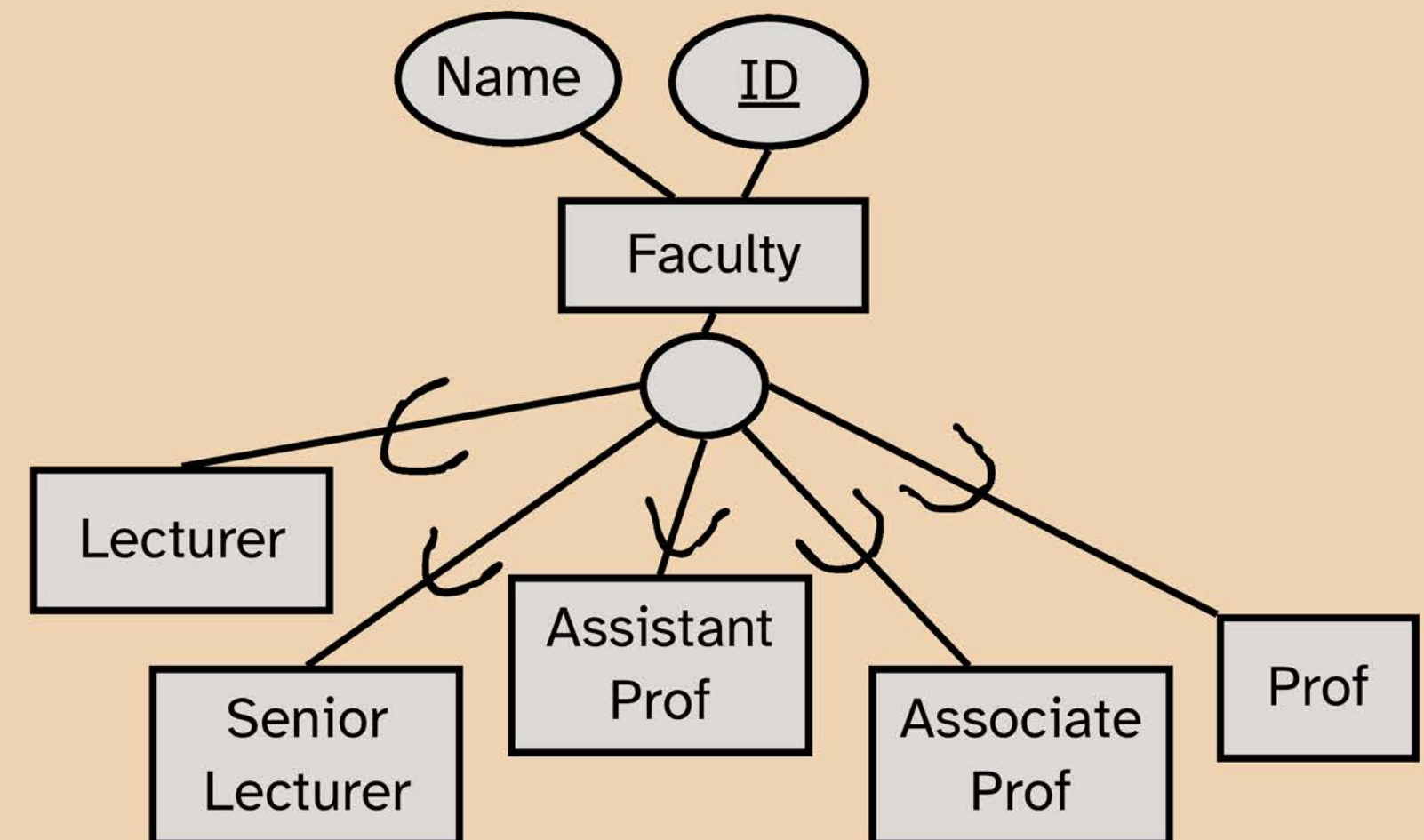
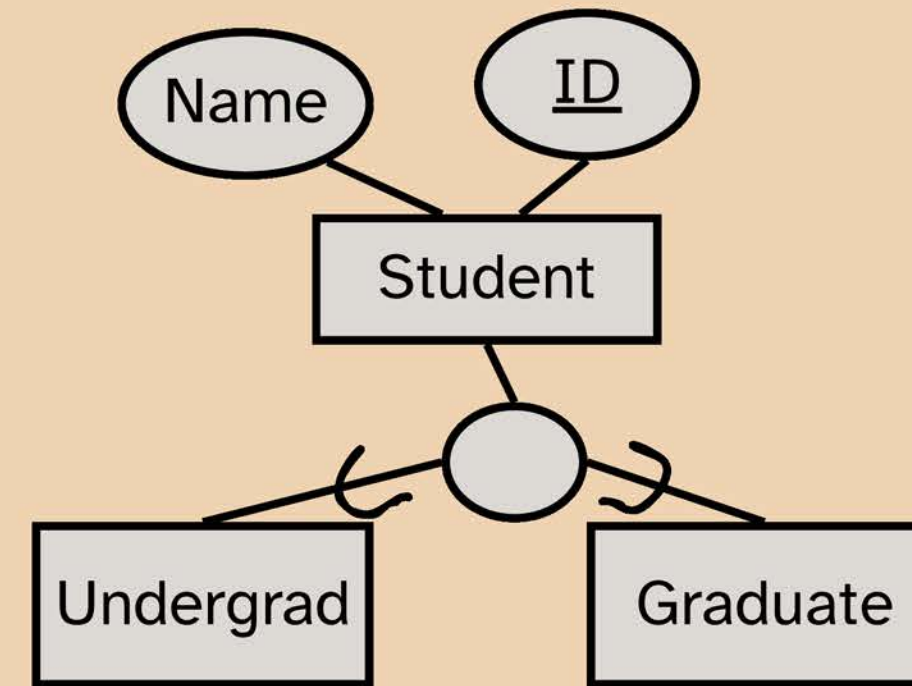
Enhanced Entity Relationship Modeling

EER (Enhanced Entity-Relationship) modeling is used to extend/enhance the traditional ER model to better represent more complex scenarios. EER captures more nuanced relationships and constraints.

- ✦ Includes all modeling concepts of basic ER
- ✦ Used to model data more completely and accurately
- ✦ Additional concepts – subclass/superclass, inheritance, specialization/generalization, categories

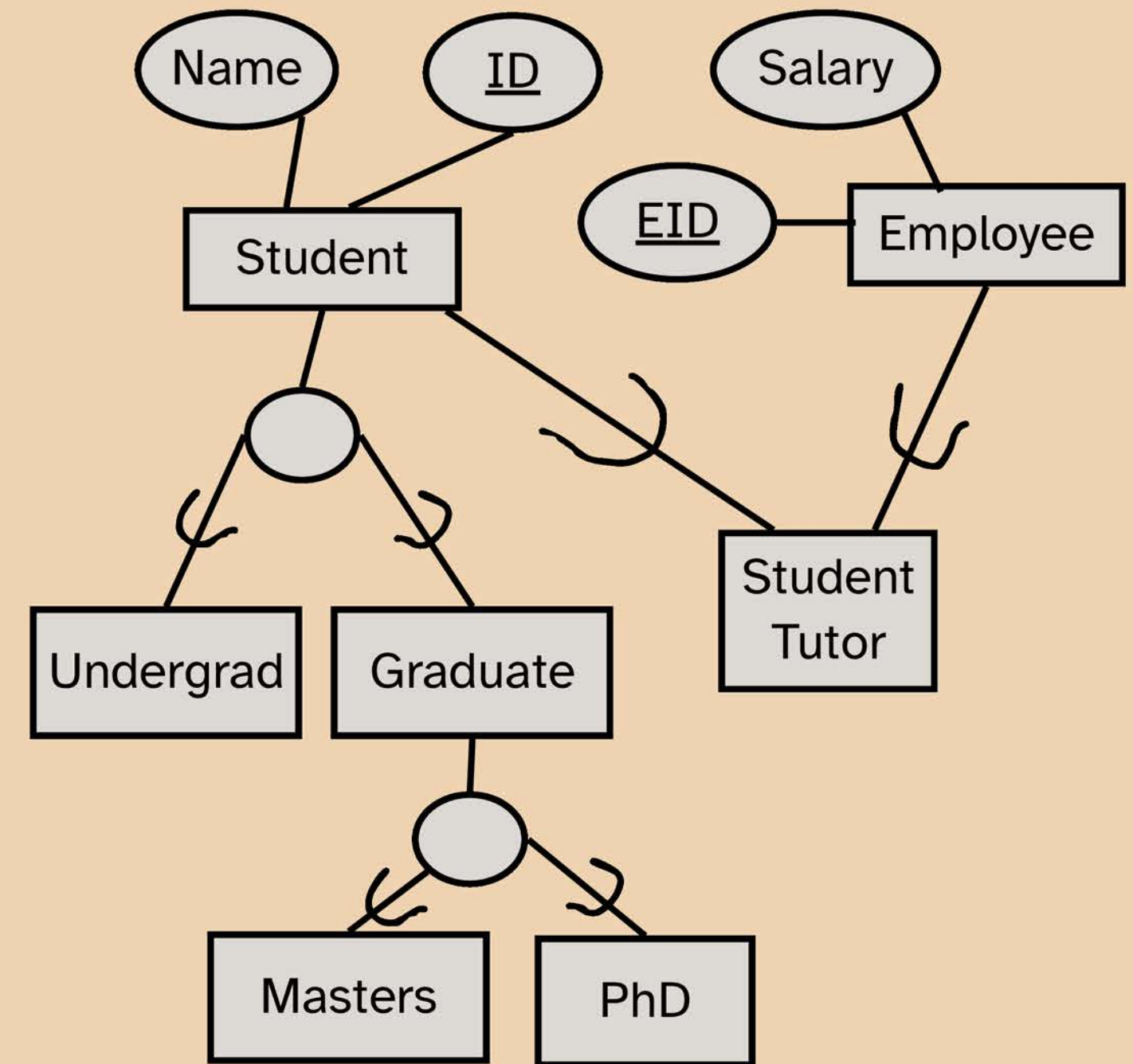
Subclasses & Superclasses (1)

- An entity type can have meaningful sub-groupings of its entities. EER diagrams extend ER diagrams to represent these additional subgroupings.
- For example, STUDENTS in a 'University' mini-world can be divided into two sub-groups: Undergraduate and Graduate students based on their level of degree. FACULTY members can be grouped according to their designations: Lecturer, Senior Lecturer, Assistant Professor etc.
- Here STUDENT and FACULTY are superclass, while the Undergrad/Graduate and Lecture/Professor/etc are their subclasses respectively.



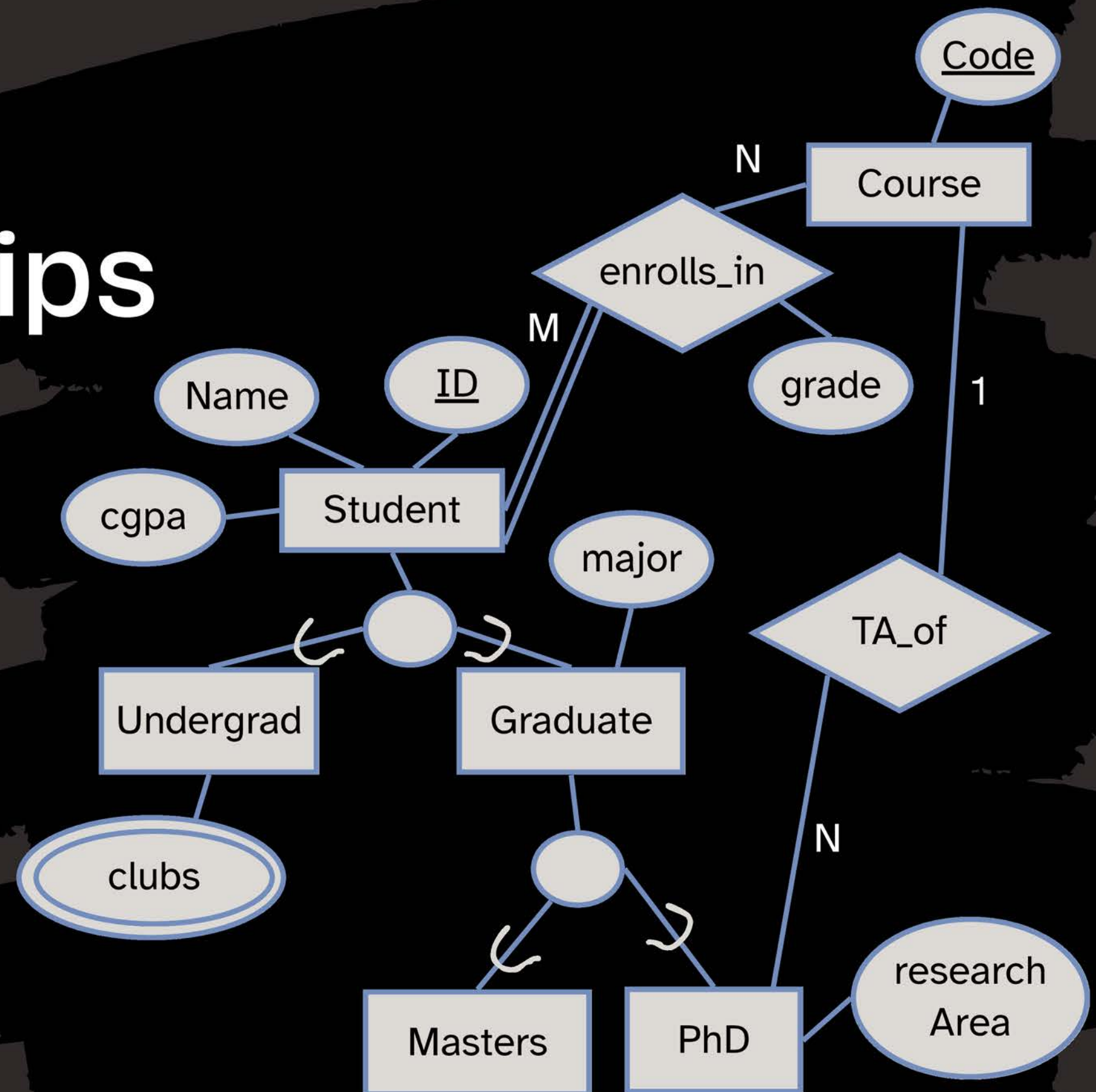
Subclasses & Superclasses (2)

- The subclasses can have further subclasses, thus forming a hierarchy or tree structure, e.g. Graduate Students can be divided into Masters and PhD students.
- A subclass may be shared by multiple superclasses, thus forming a lattice structure, for example, Student Tutor can be a subclass of both Student and Employee.
- Subclass/Superclass relationships are also called IS-A relationship, e.g. an undergrad student IS-A student or an assistant professor IS-A faculty member.
- If an entity is a member of any subclass, they MUST also belong to the superclass, e.g. a Lecturer MUST also belong to the Faculty superclass entity type. However, a member of the superclass, may belong in several or may not belong in any of the subclasses.



Inheritance & Local Attributes/Relationships

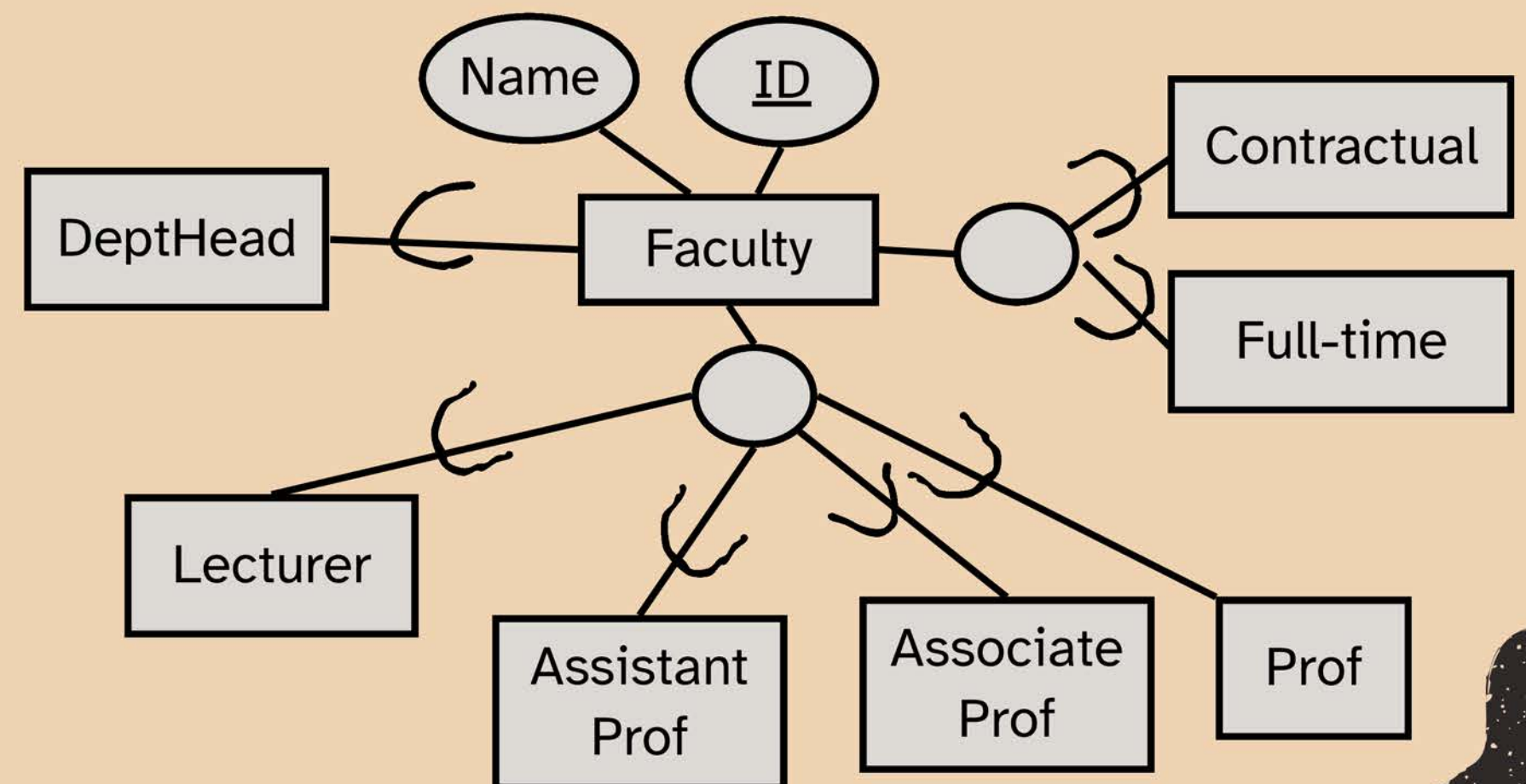
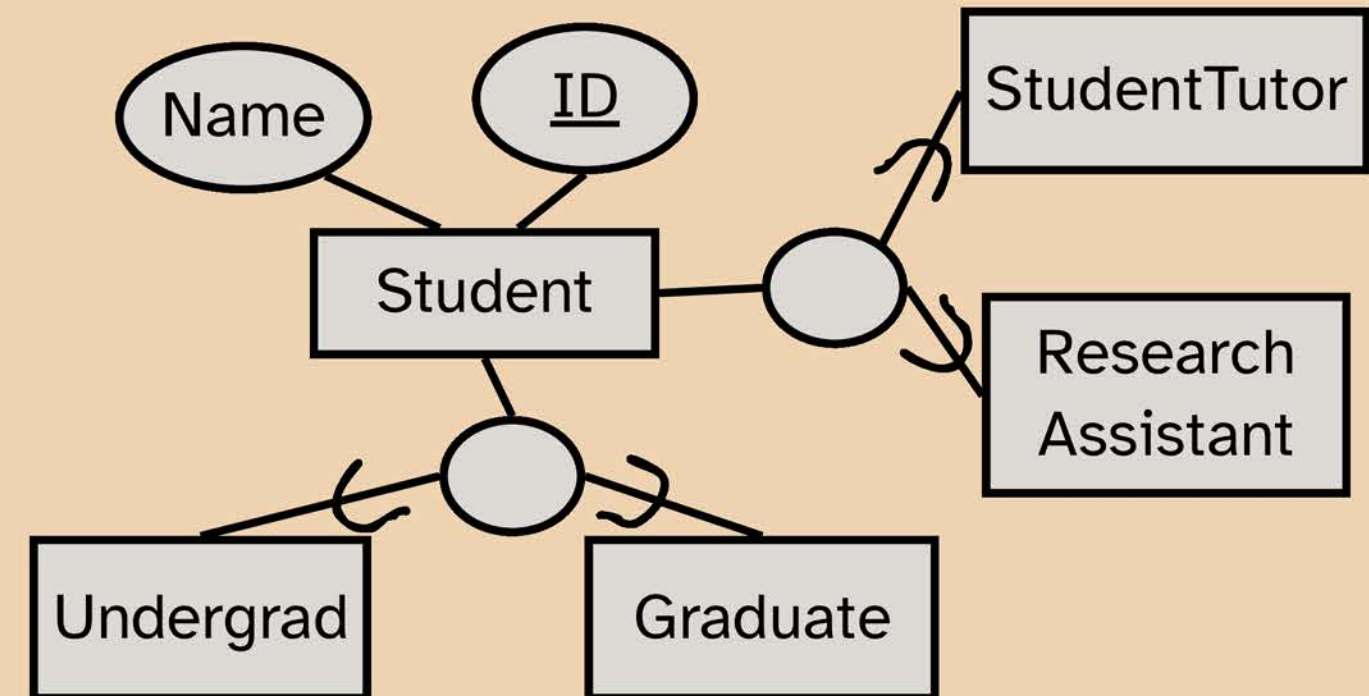
- A subclass entity will inherit all attributes and all relationships of its superclass. So, ID, name, cgpa are inherited by all Undergrad and Graduate Student. Also, the enrolls_in relationship is inherited by all the subclasses. Masters/PhD students will inherit attributes & relationships from both Student and Graduate.
- Subclasses can have local/specific attributes (e.g. clubs/researchArea) and local/specific relationships (e.g. TA_of).



Specialization

Specialization is the process of defining a set of subclasses of a superclass

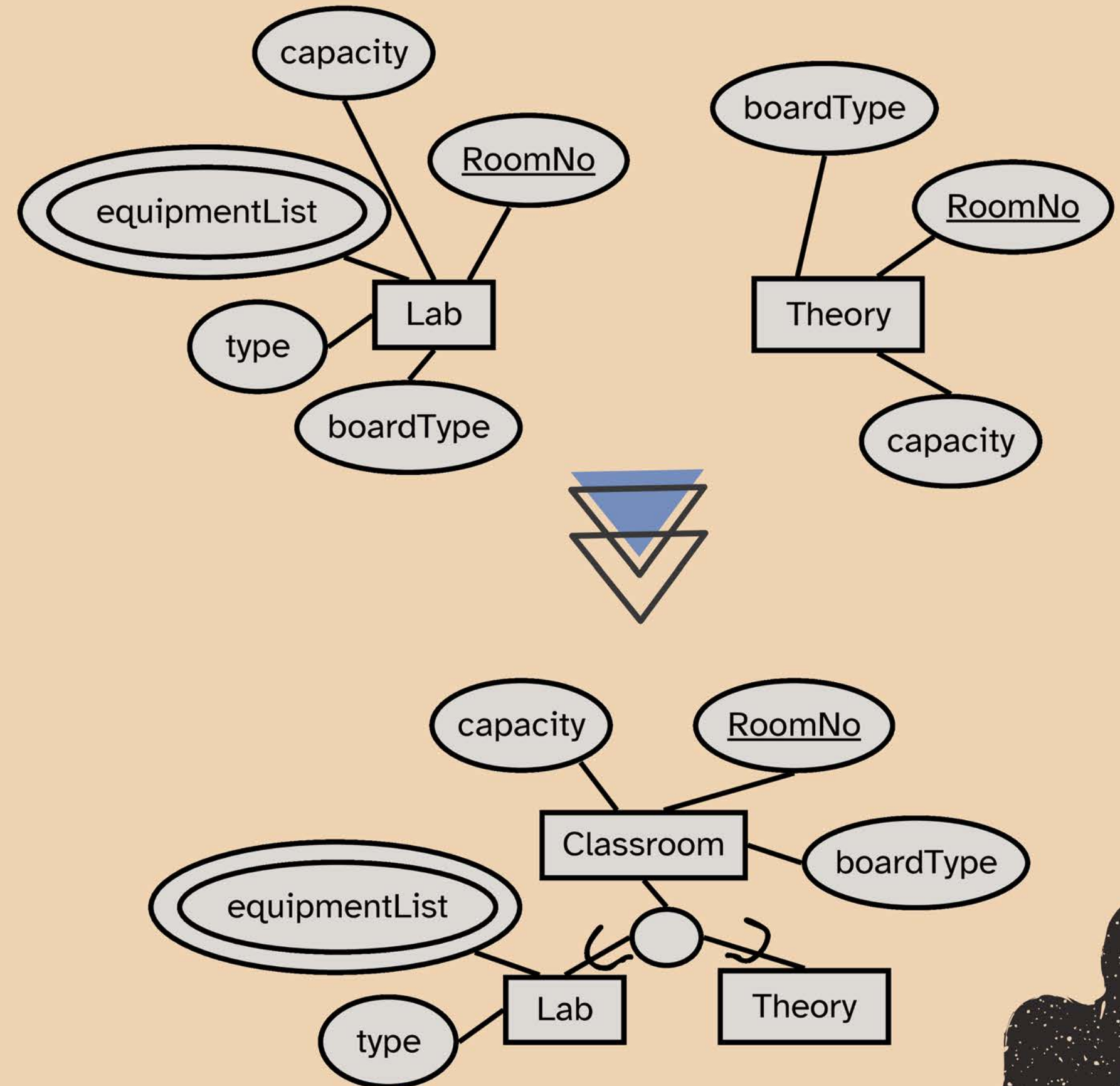
- The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass. E.g. {Undergrad, Graduate} is a specialization of Student based on their academic degree.
- There may be several specialization of the same superclass. E.g. Student has another specialization based on student jobs: {Student Tutor, Research Assistant}.
- In the given example, Faculty has three specializations: one based on type of contract with 2 subclasses, another based on designation with 4 subclasses and the third one based on whether the Faculty is department head or not with 1 subclass in that specialization.



Generalization

Generalization is the opposite of specialization- it is the process of defining a superclass from a set of subclasses .

- Several entity types with many common attributes can be generalized into a superclass. The original entity types then become the subclass of the generalized superclass.
- While converting an ER to an EER, a database designer may use the process of specialization or generalization or both according to their need. It is not possible to determine which was used from a given EER, e.g., Car and Truck can be generalized into Vehicles or Vehicles can be specialized into Car and Truck.
- In the given example of the University mini-world, the two entity types TheoryRoom and LabRoom have several common attributes. So they are generalized to form a superclass- Classroom. All common attributes and common relationships (if any) will be added to the superclass and removed from the subclasses. The attributes/relationships that are not common will remain in the subclasses.



Constraints on Specialization/Generalization (1)

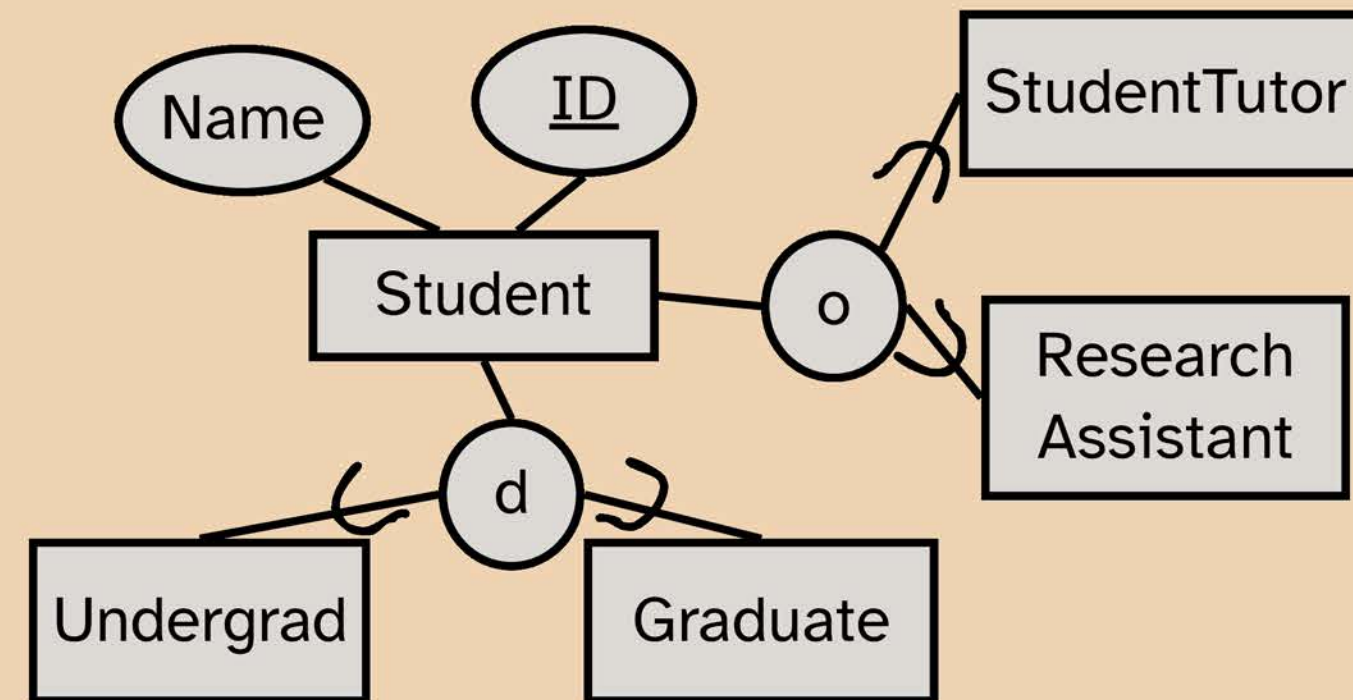
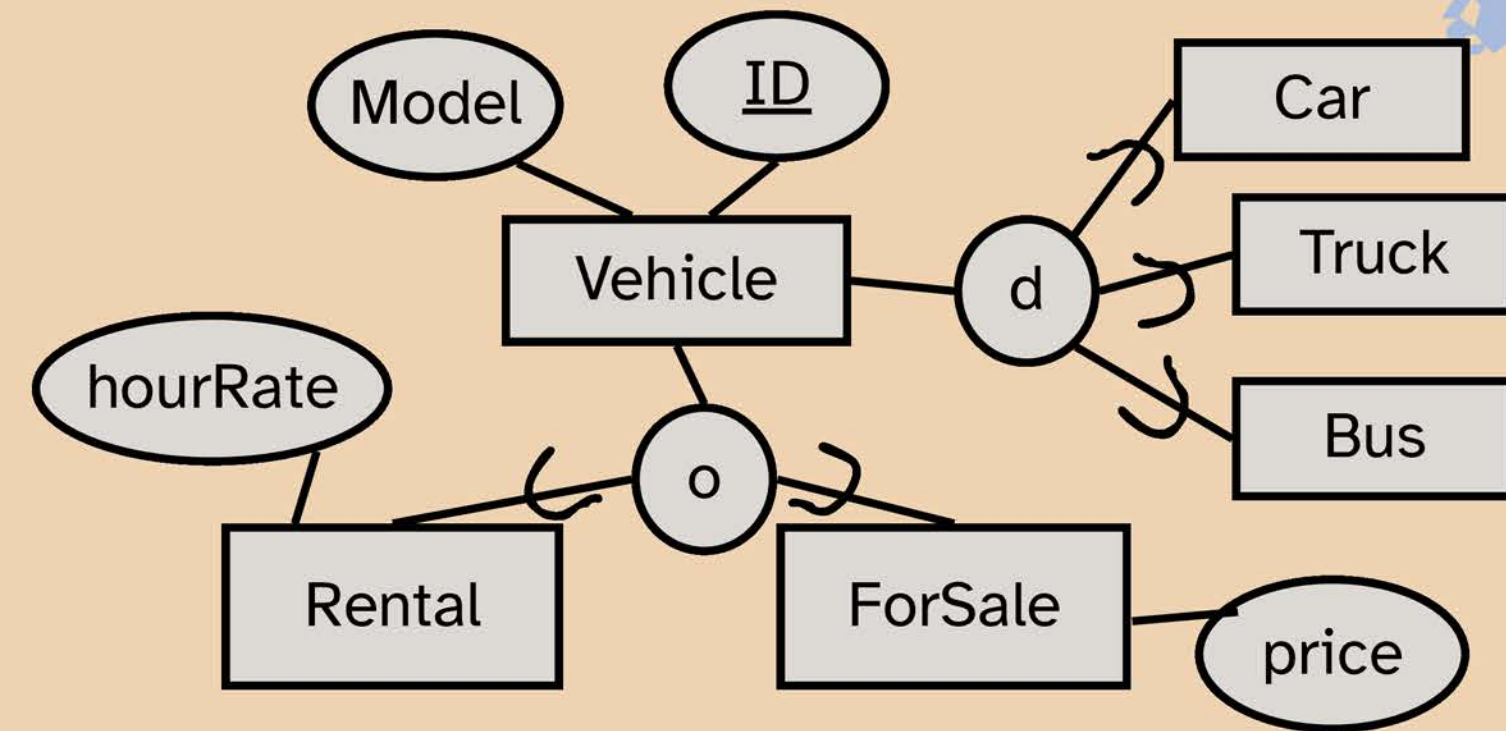
★ **Disjointness Constraint** indicates the maximum number of subclasses an entity can be a member of. There are two options, a specialization/generalization can be:

d → not both

➤ **Disjoint:** An entity can be a member of maximum 1 subclass of that specialization. It is shown using a 'd' in the EER diagram. For example, a vehicle can either be a car or a bus or a truck, or a student can either belong to the undergrad subclass or graduate subclass but not both.

➤ **Overlapping:** An entity can be a member of more than one subclass within the same specialization. It is shown using an 'o' in the EER diagram. For example, a student can belong to both student tutor and research assistant subclass at the same time, or the same vehicle can be available for sale and for renting both.

o → both ✓



Constraints on Specialization/Generalization (2)

★ **Completeness Constraint** indicates the minimum number of subclasses an entity must be a member of. There are two options, a specialization/generalization can be:

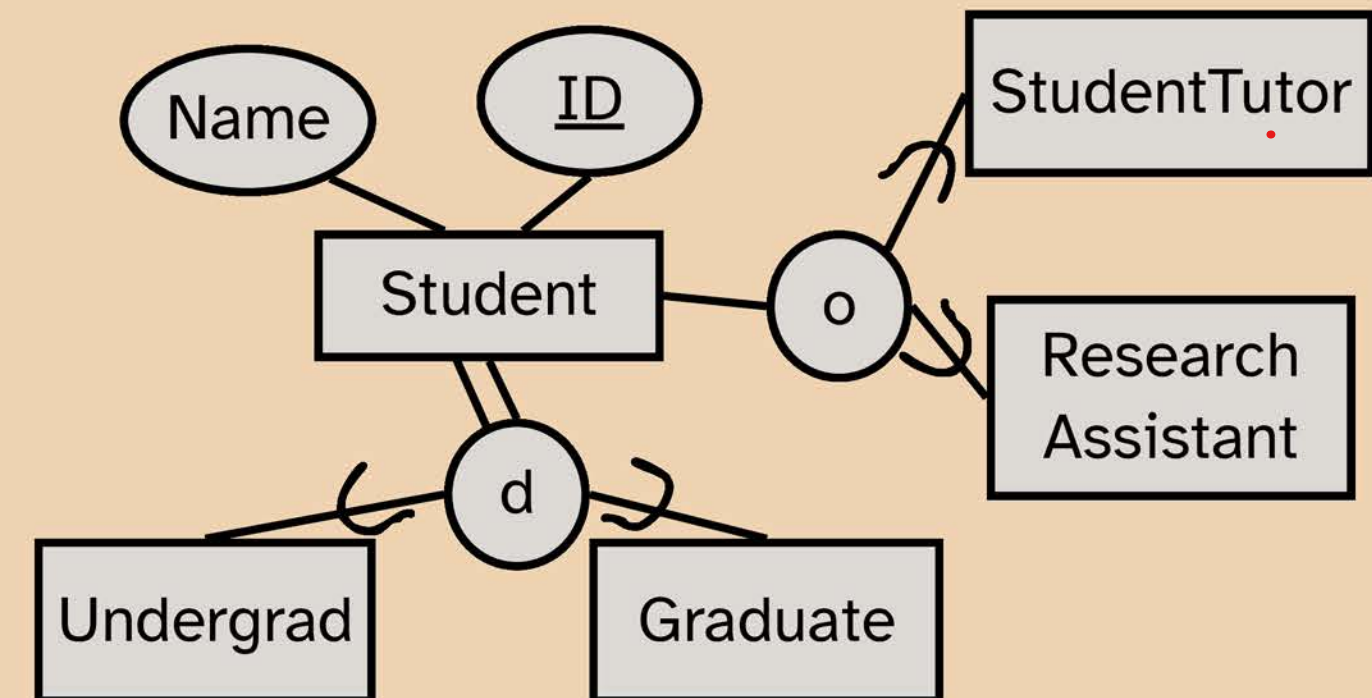
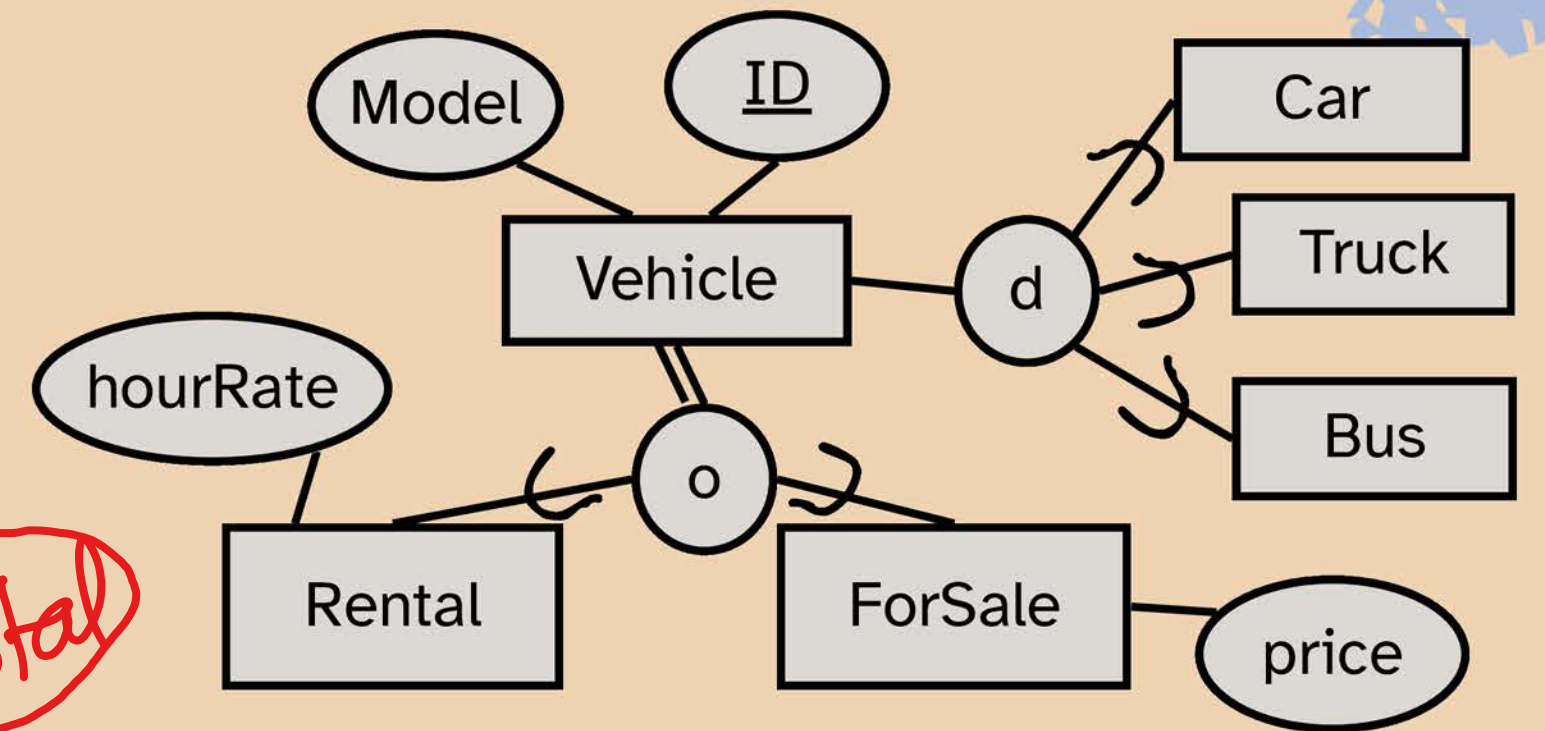
➤ **Total:** An entity must be a member of at least 1 subclass of that specialization. It is shown using a double line in the EER diagram. For example, every student must be either undergrad or graduate student.

must participate = total

➤ **Partial:** An entity may not belong to any of the subclasses within the specialization. It is shown using a single line in the EER diagram. For example, a student can belong to the student tutor or research assistant subclass or may not belong to either as all students will not have student jobs.

Can/cant not must

Thus we have 4 possible specialization/generalization: disjoint-total, disjoint-partial, overlapping-total, overlapping-partial.

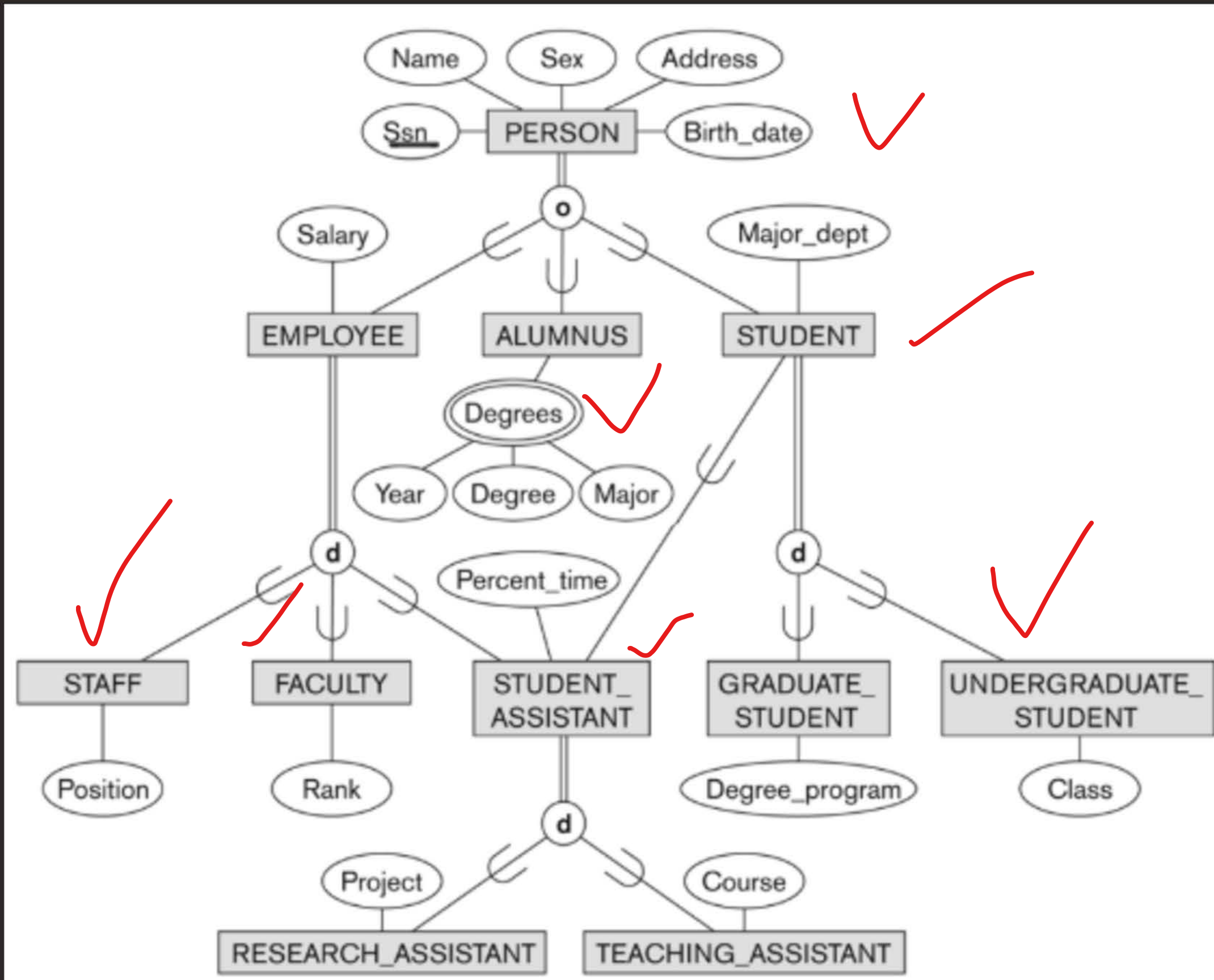


Applying EER Concepts in Practice (1)

Given the following data requirements for a University database management system, construct an EER diagram:

- There are only 3 types of persons in the university: Employee, Alumnus and Student. Every person has a unique SSN, Name, Sex, Address, Birth date. Employees have a salary attribute and students have a major attribute. ~~Alumnus has several degrees composed of year, institute and major.~~ A person can be of different types at the same time, e.g. an employee can also be an alumni member.
 - Employees must belong to only 1 type: Staff (having attribute Position), Faculty (having attribute Rank) and Student Assistant (having attribute Percent_Time). Student Assistant also inherits from Students.
 - Students can be in any one of only two types: undergraduate student or graduate student. Graduate students have Degree_program and undergraduate students have class attributes.
 - Student Assistants can be any one of only two types: Research and Teaching assistant with attributes project and course respectively.
- 

Applying EER Concepts in Practice (2)



Applying EER Concepts in Practice (3)

Design an EER diagram for a School Management System. The school wants to keep track of all their employees, students, clubs, courses, grades etc. They have hired you to help them design the database for that purpose.

You can design your EER as you wish, but it must satisfy the following constraints:

- there should be at least one disjoint-total specialization/generalization.
- There should be at least five regular/strong entities [excluding subclasses]
- There must be a recursive relationship.
- There must be at least one 1:1 relationship.
- There must be at least one multivalued-composite attribute.

Show all important attributes, entities and relationships in order to represent a clear and complete scenario. The EER diagram should be logically accurate and realistic, representing the database of the given scenario.

This is a open-ended design question, different people should have different solutions.
Try it yourself.



WHAT NEXT?

Lecture 4: Relational Database Constraints



LOADING.....

